

Obesity Level Classification

Machine Learning Final Project — Technical Documentation

Project	Estimation of Obesity Levels Based on Eating Habits and Physical Condition
Organization	BNCC RnD (Bina Nusantara Computer Club Research and Development)
Task Type	Supervised multi-class classification
Target Variable	NObeyesdad (7 obesity level categories)
Best Model	Random Forest (tuned) — Macro F1 = 0.90, Accuracy = 0.91
Deployment	Streamlit web application (app.py)

1. Project Overview

This project builds a machine learning pipeline that estimates a person's obesity level from anthropometric measurements (age, height, weight) and lifestyle-related habits (eating patterns, physical activity, transportation, smoking, alcohol consumption, and screen time). The target variable, **NObeyesdad**, has seven ordered categories ranging from Insufficient Weight to Obesity Type III.

The workflow covers data cleaning, exploratory data analysis (EDA), preprocessing, training and tuning three candidate classifiers, evaluating them with cross-validated and held-out metrics, and packaging the best-performing model into an interactive Streamlit application for demonstration.

1.1 Objectives

- Explore and clean the raw survey dataset, handling missing values appropriately.
 - Build a reusable preprocessing pipeline (scaling, ordinal encoding, one-hot encoding).
 - Train and compare KNN, Random Forest, and Gradient Boosting classifiers.
 - Tune hyperparameters with cross-validated grid search and select the best model by macro F1-score.
 - Evaluate the final model on a held-out test set and analyze misclassification patterns.
 - Deploy the trained model as an interactive Streamlit web app for end users.
-

2. Dataset

The dataset, `estimation_of_obesity_levels_based_on_habits.csv`, contains self-reported anthropometric and lifestyle attributes for individuals, sourced from a survey-style obesity study. It has 17 columns: 16 predictor features and 1 target label.

2.1 Feature Description

Column	Type	Description
Gender	Categorical	Female / Male
Age	Numeric	Age in years
Height	Numeric	Height in meters
Weight	Numeric	Weight in kilograms
family_history_with_overweight	Categorical	Family history of being overweight (yes/no)
FAVC	Categorical	Frequent consumption of high-caloric food (yes/no)
FCVC	Numeric	Frequency of vegetable consumption (0–3)
NCP	Numeric	Number of main meals per day
CAEC	Ordinal	Eating between meals: no / Sometimes / Frequently / Always
SMOKE	Categorical	Smoking habit (yes/no)
CH2O	Numeric	Daily water consumption (liters, 0–5 scale)

Column	Type	Description
SCC	Categorical	Monitors daily calorie intake (yes/no)
FAF	Numeric	Physical activity frequency (0–5 scale)
TUE	Numeric	Time using technology devices (0–5 scale)
CALC	Ordinal	Alcohol consumption: no / Sometimes / Frequently / Always
MTRANS	Categorical	Main transportation mode
NObeyesdad	Target (7 classes)	Obesity level — the label to predict

2.2 Missing Data

An initial scan showed 211 missing rows across most non-demographic columns (Height, Weight, FAVC, FCVC, NCP, CAEC, SMOKE, CH2O, SCC, FAF, TUE, CALC, MTRANS, and the NObeyesdad label itself), while Gender and Age had no missing values. Rows missing the target label were dropped, since a label cannot be imputed for a supervised task. Remaining missing values were imputed as follows:

- **Numeric columns** (Height, Weight, FCVC, NCP, CH2O, FAF, TUE): mean imputation for the initial pass; the modeling pipeline itself uses median imputation.
- **Categorical columns** (family_history_with_overweight, FAVC, CAEC, SMOKE, SCC, CALC, MTRANS): most-frequent-value imputation.

After cleaning, the dataset had zero missing values in every column.

2.3 Target Class Distribution

The seven obesity classes are reasonably balanced after removing rows with a missing label, with counts ranging roughly from 240 to 320 per class (no class dominates the dataset), which is favorable for macro-averaged evaluation metrics.

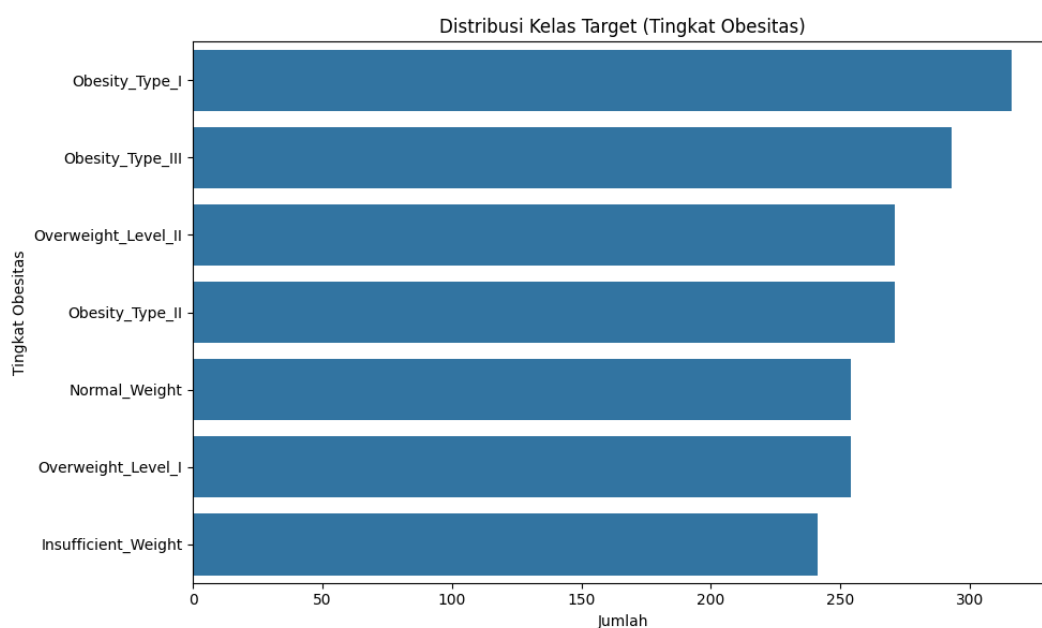


Figure 1. Distribution of the target classes (Tingkat Obesitas / Obesity Level).

2.4 Feature Correlation

A correlation heatmap of the numeric features was inspected to check for strong linear relationships and potential redundancy before modeling.

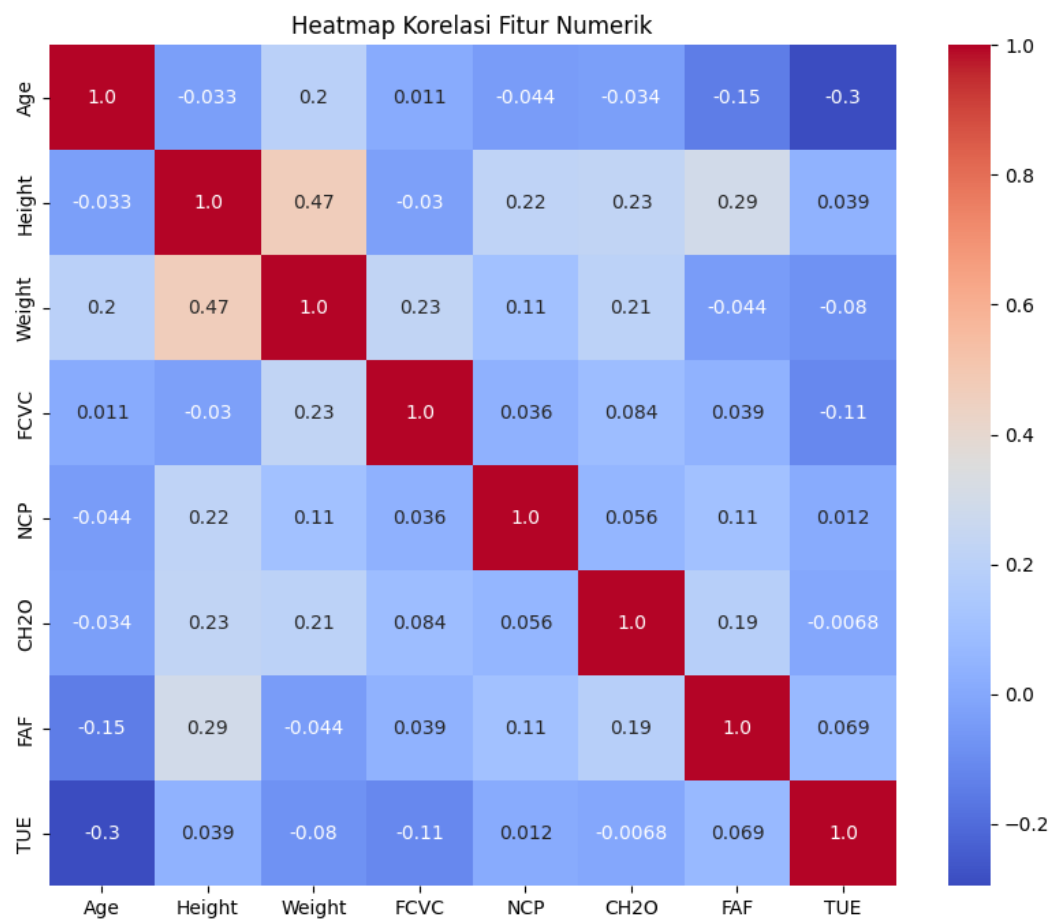


Figure 2. Correlation heatmap of numeric features.

2.5 Meal Frequency vs. Obesity Level

A boxplot of the number of main meals per day (NCP) grouped by obesity level was used to spot behavioral trends across classes during EDA.

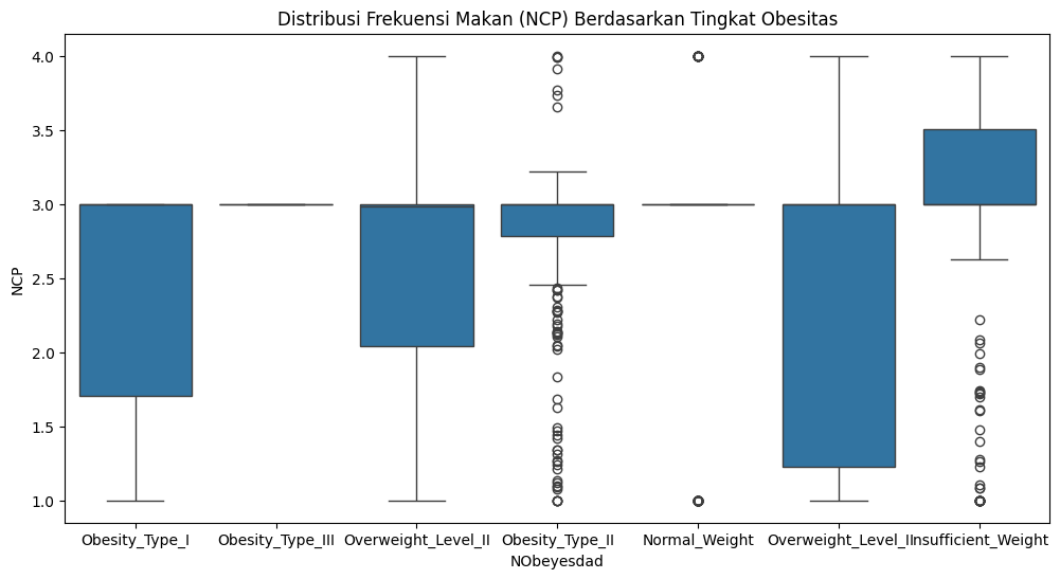


Figure 3. Distribution of meal frequency (NCP) across obesity levels.

2.6 Class Balance Check (post-cleaning)

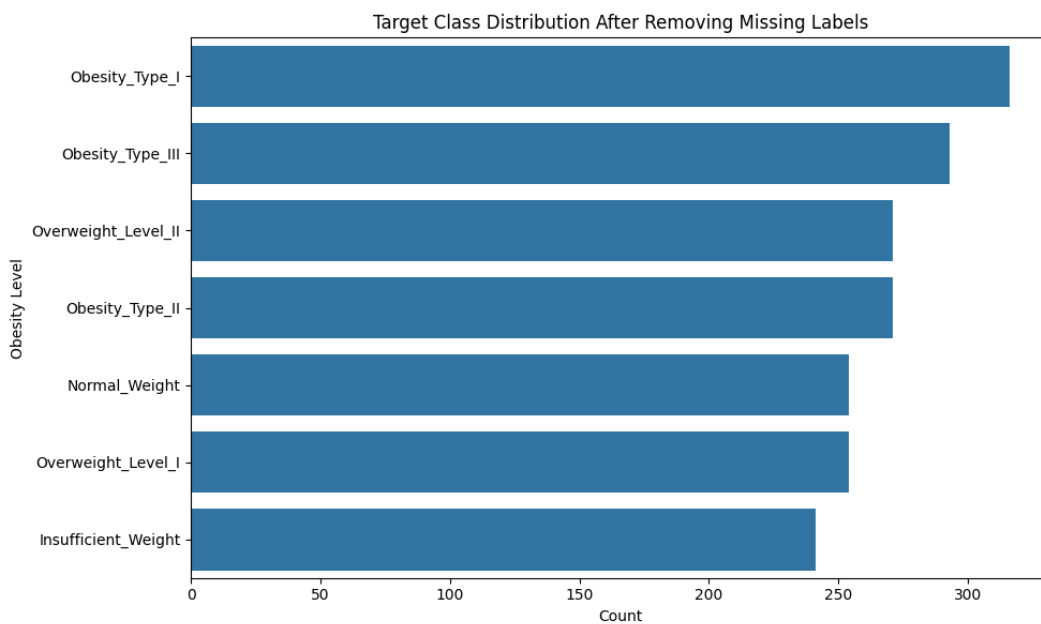


Figure 4. Target class distribution after removing rows with a missing label.

3. Preprocessing & Modeling Pipeline

3.1 Feature Groups

Features were split into three groups, each handled by a dedicated transformer inside a scikit-learn ColumnTransformer:

Group	Columns	Transformer
Numeric	Age, Height, Weight, FCVC, NCP, CH2O, FAF, TUE	Median imputation → StandardScaler
Ordinal	CAEC, CALC	Most-frequent imputation → OrdinalEncoder (no < Sometimes < Frequently < Always)
Nominal	Gender, family_history_with_overweight, FAVC, SMOKE, SCC, MTRANS	Most-frequent imputation → OneHotEncoder (drop='if_binary')

3.2 Train / Test Split

The cleaned dataset was split 80/20 into training and test sets using `train_test_split` with `random_state=42` and `stratify=y`, preserving the class proportions in both sets.

3.3 Candidate Models

- **K-Nearest Neighbors (KNN)** — baseline distance-based classifier.
- **Random Forest** — bagged ensemble of decision trees.
- **Gradient Boosting** — sequential boosted ensemble of decision trees.

3.4 Baseline Results (default hyperparameters)

Model	Test Accuracy
KNN (Baseline)	0.7500
Random Forest	0.8947
Gradient Boosting	0.8868

Even before tuning, both ensemble methods clearly outperformed the KNN baseline, motivating a focus on Random Forest and Gradient Boosting for hyperparameter tuning.

3.5 5-Fold Stratified Cross-Validation

Each candidate pipeline was evaluated with 5-fold stratified cross-validation (`StratifiedKFold`, shuffled, `random_state=42`) on the training set, scoring accuracy, macro precision, macro recall, and macro F1 to get a more robust performance estimate than a single split.

3.6 Hyperparameter Tuning (GridSearchCV)

Each model was tuned with `GridSearchCV` using the same 5-fold stratified CV, optimizing for macro F1-score. The search grids and best results were:

Model	Grid Searched	Best CV F1	Best Parameters
KNN	n_neighbors: [3,5,7,9,11] weights: [uniform, distance] metric: [euclidean, manhattan]	0.8269	metric=manhattan n_neighbors=5 weights=distance
Random Forest	n_estimators: [200,400] max_depth: [None,10,20] min_samples_split: [2,5] min_samples_leaf: [1,2]	0.8998	max_depth=20 min_samples_leaf=1 min_samples_split=2 n_estimators=200

Model	Grid Searched	Best CV F1	Best Parameters
Gradient Boosting	n_estimators: [100,200] learning_rate: [0.05,0.1] max_depth: [2,3] subsample: [0.8,1.0]	0.8960	learning_rate=0.1 max_depth=3 n_estimators=200 subsample=0.8

4. Model Evaluation

All tuned models were evaluated on the held-out 20% test set (380 samples). **Random Forest was selected as the best model**, based on the highest macro F1-score.

4.1 Test Set Performance Summary

Final test-set metrics (accuracy, and macro-averaged precision / recall / F1) for the tuned Random Forest, the selected best model:

Metric	Value
Accuracy	0.91
Precision (macro avg)	0.91
Recall (macro avg)	0.90
F1-score (macro avg)	0.90

4.2 Per-Class Classification Report (Random Forest)

Class	Precision	Recall	F1-score	Support
Insufficient_Weight	0.96	0.96	0.96	48
Normal_Weight	0.71	0.86	0.78	51
Overweight_Level_I	0.88	0.82	0.85	51
Overweight_Level_II	0.90	0.81	0.85	54
Obesity_Type_I	0.98	0.89	0.93	63
Obesity_Type_II	0.95	0.98	0.96	54
Obesity_Type_III	0.98	1.00	0.99	59
accuracy			0.91	380
macro avg	0.91	0.90	0.90	380
weighted avg	0.91	0.91	0.91	380

The model performs strongly across nearly all classes ($F1 \geq 0.85$), with **Normal_Weight** being the weakest class (precision 0.71) — it is most often confused with adjacent, visually and behaviorally similar categories.

4.3 Confusion Matrix

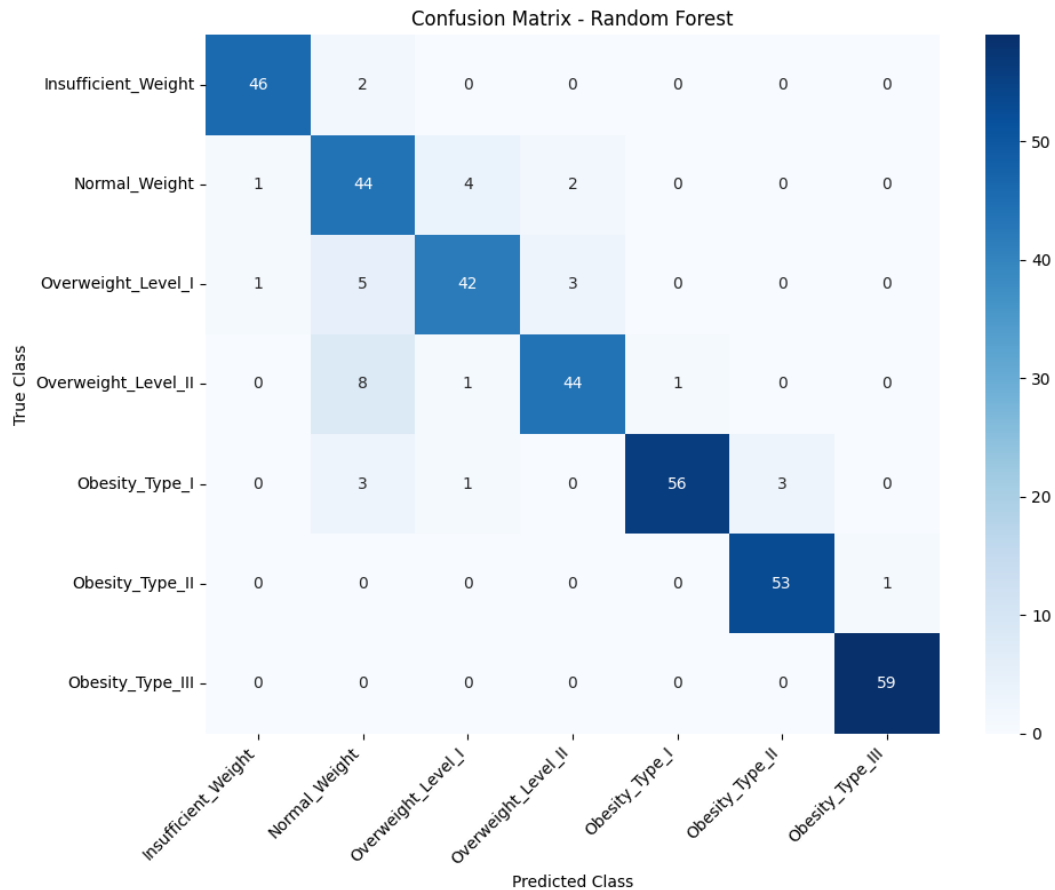


Figure 5. Confusion matrix of the tuned Random Forest on the test set.

The largest single source of error (8 test samples) is **Overweight_Level_II** being misclassified as **Normal_Weight**, consistent with the lower precision/recall seen for the Normal_Weight class — these two categories sit close together on the underlying weight/BMI spectrum.

5. Model Deployment

5.1 Model Persistence

The best pipeline (preprocessing + tuned Random Forest) is serialized with `joblib` to `best_obesity_pipeline.joblib`. This single artifact bundles imputation, scaling, encoding, and the trained classifier, so raw, unprocessed input can be passed to it directly at inference time.

5.2 Streamlit Application

The notebook auto-generates a self-contained Streamlit app (`app.py`) that loads the saved pipeline and exposes a form-based UI for entering an individual's personal information, eating habits, and lifestyle attributes. On submission, it:

- Assembles the form inputs into a single-row DataFrame matching the training schema.
- Runs the loaded pipeline's `predict()` to get the predicted obesity class.
- Displays the predicted label, plus the individual's BMI as supplementary context (BMI is shown for reference only and is not a model feature).
- Shows the raw input data in an expandable panel for transparency.

- Includes an academic-project disclaimer clarifying the app is not a medical diagnosis tool.

5.3 Generated Deployment Files

File	Purpose
app.py	Streamlit web application (form UI + prediction logic)
best_obesity_pipeline.joblib	Serialized preprocessing + Random Forest pipeline
requirements.txt	Python dependencies for running the app
README.md	Project usage instructions (see the accompanying README)

6. How to Reproduce / Run

6.1 Re-running the Notebook

- Open `BNCC_FinalProject_Altair.ipynb` in Jupyter/Colab.
- Ensure `estimation_of_obesity_levels_based_on_habits.csv` is in the working directory.
- Run all cells top to bottom: EDA → cleaning → preprocessing → baseline models → cross-validation → grid search tuning → evaluation → model export → app generation.

6.2 Running the Streamlit App Locally

```
pip install -r requirements.txt
streamlit run app.py
```

The app expects `best_obesity_pipeline.joblib` to be in the same folder as `app.py`.

7. Limitations & Notes

- The dataset is self-reported survey data; some features (e.g., FCVC, NCP, FAF) are coarse numeric scales rather than precise measurements.
- About 211 rows had missing values and were imputed (mean / most-frequent); this can smooth out some natural variance in the data.
- The `Normal_Weight` and `Overweight_Level_II` classes show the most confusion, likely due to overlapping lifestyle profiles near the class boundary.
- The Streamlit app is built for demonstration purposes only and explicitly disclaims medical use.

8. Project Structure

<code>BNCC_FinalProject_Altair.ipynb</code>	Main analysis notebook (EDA -> training -> export)
<code>estimation_of_obesity_levels_based_on_habits.csv</code>	Raw dataset (not included in repo)
<code>app.py</code>	Generated Streamlit application
<code>best_obesity_pipeline.joblib</code>	Serialized trained pipeline
<code>requirements.txt</code>	Python dependencies
<code>README.md</code>	Project documentation

